

LAB 2658 Accelerating AI-Ready Data Pipelines with Medallion Architecture Accelerator on watsonx.data

Hands-on lab guide

Authors

Yin Chen

Chris Elder

Albert Grankin



Table of Contents

- 1 Introduction 5
 - 1.1 The Dataset 5
 - 1.2 Medallion Architecture 5
 - 1.3 About this hands-on lab 6
- 2 Work Station Setup 6
 - 2.1 Configure the working environment 6
- 3 Build an Orchestration Pipeline for Medallion Data 9
 - 3.1 Generate an API Key 9
 - 3.2 Set up the project in Cloud Pak for Data 10
 - 3.3 Create environment variables for the spark job..... 14
 - 3.4 Create the spark jobs 14
 - 3.5 Create an Orchestration Pipeline for the Spark Jobs 20
- 4 Load Medallion Data using CLI tools (Optional) 27
 - 4.1 Configure the cli directory 27
- 5 Verify the Medallion Tables 32
 - 5.1 Verify one of the medallion tables..... 32
- 6 Automated Registration of Medallion Tables with IBM Knowledge Catalog (Demo) 36
- 7 Confluent Integration with Medallion (Demo)..... 36
- 8 Feedback section..... 36
- 9 Getting help and troubleshooting..... 36
 - 9.1 Reach out to the Lab instructor. 36
 - 9.2 Common troubleshooting tips 36
 - 9.3 Getting help..... 36

10 Lab Reference Materials (for instructor)..... 36

Notices and disclaimers

© 2026 International Business Machines Corporation. No part of this document may be reproduced or transmitted in any form without written permission from IBM.

U.S. Government Users Restricted Rights — use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM.

This document is current as of the initial date of publication and may be changed by IBM at any time. Not all offerings are available in every country in which IBM operates.

Information in these presentations (including information relating to products that have not yet been announced by IBM) has been reviewed for accuracy as of the date of initial publication and could include unintentional technical or typographical errors. IBM shall have no responsibility to update this information.

This document is distributed “as is” without any warranty, either express or implied. In no event, shall IBM be liable for any damage arising from the use of this information, including but not limited to, loss of data, business interruption, loss of profit or loss of opportunity. IBM products and services are warranted per the terms and conditions of the agreements under which they are provided. The performance data and client examples cited are presented for illustrative purposes only. Actual performance results may vary depending on specific configurations and operating conditions.

IBM products are manufactured from new parts or new and used parts.
In some cases, a product may not be new and may have been previously installed. Regardless, our warranty terms apply.”

Any statements regarding IBM's future direction, intent or product plans are subject to change or withdrawal without notice.

Performance data contained herein was generally obtained in a controlled, isolated environments. Customer examples are presented as illustrations of how those customers have used IBM products and the results they may have achieved. Actual performance, cost, savings or other results in other operating environments may vary.

References in this document to IBM products, programs, or services does not imply that IBM intends to make such products, programs or services available in all countries in which IBM operates or does business.

Workshops, sessions and associated materials may have been prepared by independent session speakers, and do not necessarily reflect the views of IBM. All materials and discussions are provided for informational purposes only, and are neither intended to, nor shall constitute legal or other guidance or advice to any individual participant or their specific situation.

It is the customer’s responsibility to ensure its own compliance with legal requirements and to obtain advice of competent legal counsel as to the identification and interpretation of any relevant laws and regulatory

IBM TechXchange

requirements that may affect the customer's business and any actions the customer may need to take to comply with such laws. IBM does not provide legal advice or represent or warrant that its services or products will ensure that the customer follows any law.

Notices and disclaimers (Continued)

Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products. IBM does not warrant the quality of any third-party products, or the ability of any such third-party products to interoperate with IBM's products. IBM expressly disclaims all warranties, expressed or implied, including but not limited to, the implied warranties of merchantability and fitness for a purpose.

The provision of the information contained herein is not intended to, and does not, grant any right or license under any IBM patents, copyrights, trademarks or other intellectual property right.

IBM, the IBM logo, and ibm.com are trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at "Copyright and trademark information" at: www.ibm.com/legal/copytrade.shtml.

1 Introduction

IBM watsonx.data is an open, hybrid, and governed data lakehouse built on open standards. At its core it provides a managed Apache Spark environment that lets data engineers run large-scale transformations against Apache Iceberg tables without managing cluster infrastructure.

In this lab you will build a complete end-to-end data pipeline that demonstrates how watsonx.data Spark can be used to implement the Medallion Architecture — a layered data design pattern that progressively refines raw data into analytics-ready, high-quality tables stored in open Iceberg format.

1.1 The Dataset

The dataset in use is a subset from the CFPB Consumer Complaint Database:

<https://www.consumerfinance.gov/data-research/consumer-complaints/>

Each record represents a complaint submitted by a U.S. consumer about a financial product or service. Fields include: date received, product, sub-product, issue, company name, U.S. state, ZIP code, submission channel, company response, and a unique complaint ID.

The dataset is loaded from an S3-compatible bucket and contains hundreds of thousands of records, providing a realistic volume for demonstrating Spark's distributed processing capabilities.

1.2 Medallion Architecture

Data flows through three progressively refined layers, each stored as an Apache Iceberg tables in watsonx.data:

SOURCE

Raw CSV in S3-compatible object storage (CFPB complaints file)

BRONZE

Raw ingest — all columns as strings, minimal transformation, timestamped on arrival

SILVER

Cleansed & typed — dates cast, nulls filtered, partitioned by complaint year

GOLD

Analytics-ready — enriched with complaint year, optimized for BI queries

Each layer also produces **dimensional summary tables** — company lists, state-level complaint counts, and year-over-year summaries — making the data immediately queryable from any tool connected to watsonx.data.

1.3 About this hands-on lab

The main objective of this hands-on lab is to present an end-to-end how Medallion Architecture can be implemented in watsonx.data.

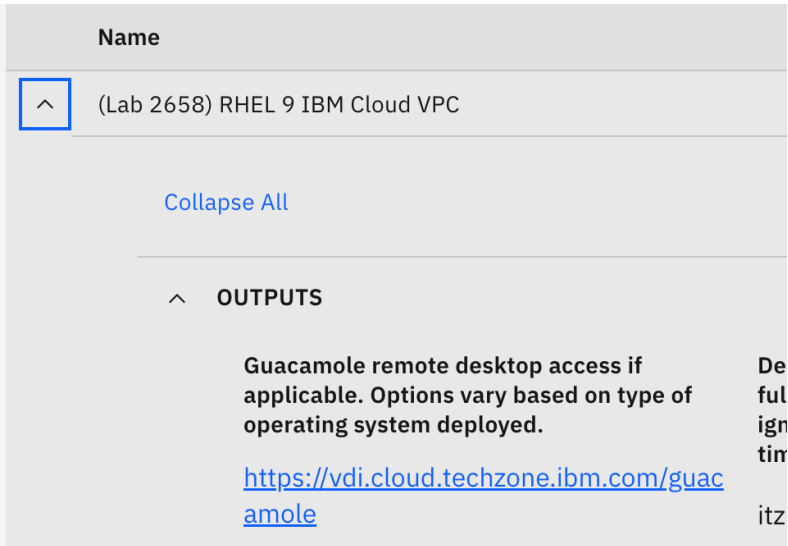
2 Work Station Setup

2.1 Configure the working environment

1. This is the github location for all lab materials:

[https://github.com/IBM/TechXchange2026/tree/main/Lab 2658 AI Ready Pipelines with Medallion Architecture](https://github.com/IBM/TechXchange2026/tree/main/Lab%202658%20AI%20Ready%20Pipelines%20with%20Medallion%20Architecture)

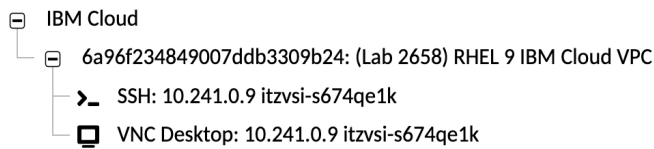
2. Open the reservation for the client and click on the Quacamole remote desktop link



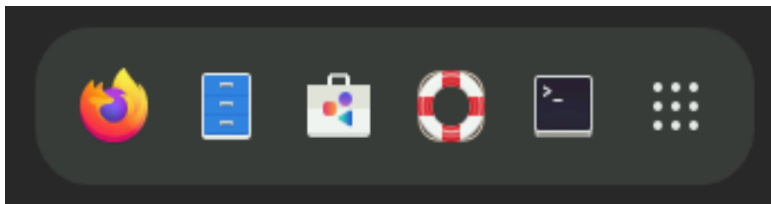
IBM TechXchange

- Click on the VNC Desktop link under ALL CONNECTIONS

ALL CONNECTIONS



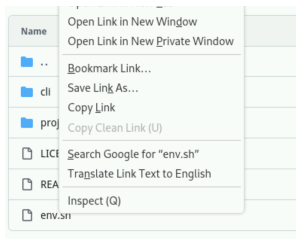
- Click on Activities in the top left corner of the screen to show the icons



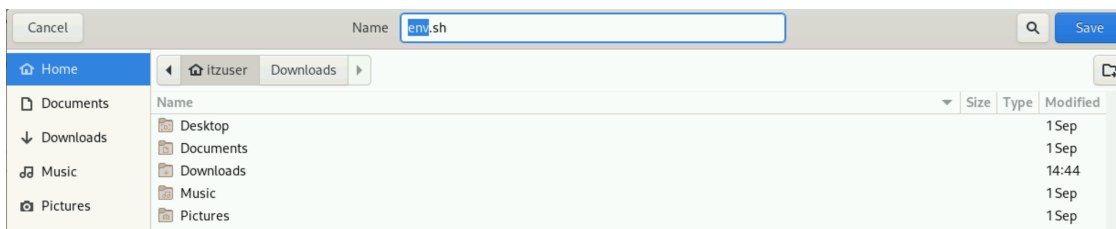
- Click on the Firefox icon and open the github location:

https://github.com/IBM/TechXchange2026/tree/main/Lab_2658_AI_Ready_Pipelines_with_Medallion_Architecture

- Hover over the env.sh, right click, and select the Save Link As...



- Click on Home and the save the env.sh file:



IBM TechXchange

8. Open on the github “project” directory and click on the LAB2658_Project.zip file. Click on the “Download raw file” icon on the right side of the screen. 



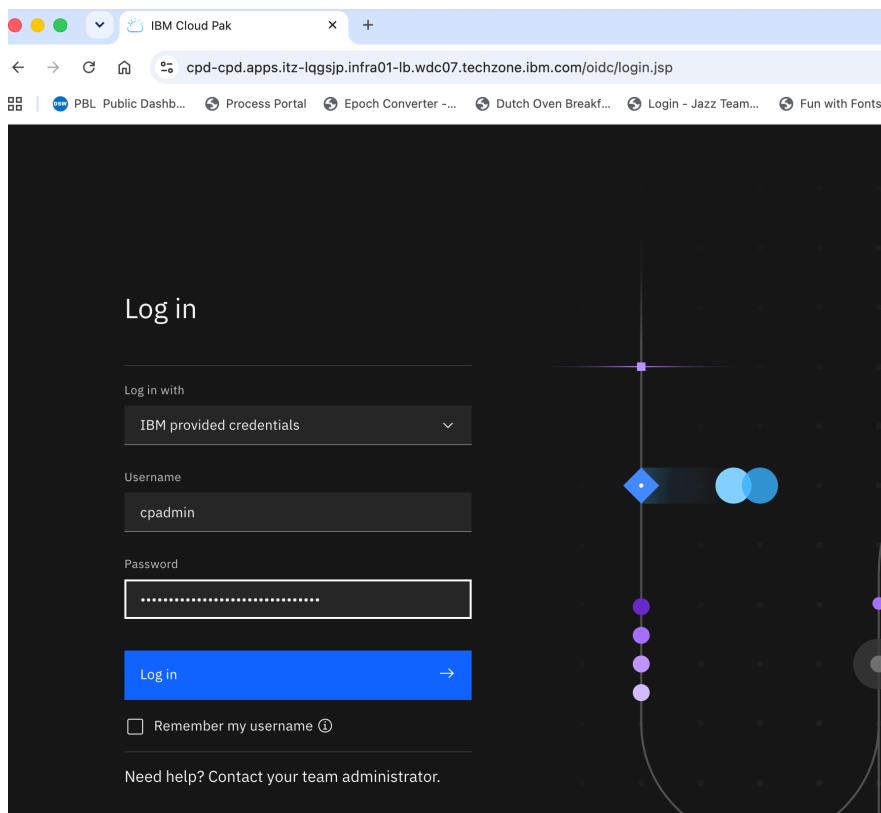
3 Build an Orchestration Pipeline for Medallion Data

3.1 Generate an API Key

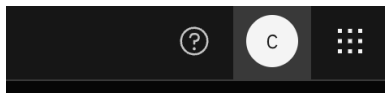
1. Log in to Cloud Pak for Data

<https://cpd-cpd.apps.itz-ry66c1.hub01-lb.techzone.ibm.com>

use credentials from the cheat sheet provided

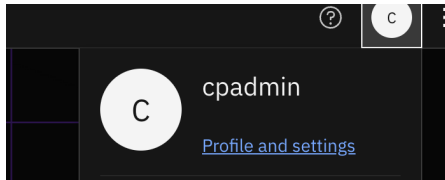


2. Click on the circular user information icon on the top right of the screen.

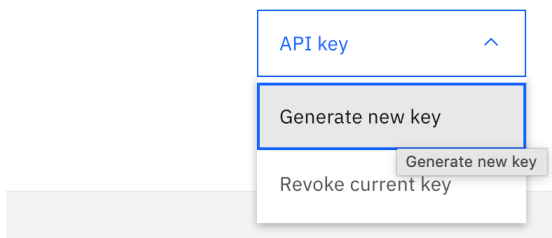


IBM TechXchange

3. Click on “Profiles and settings”



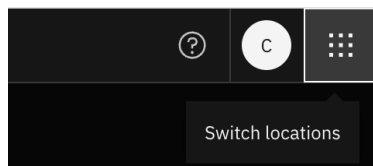
4. Click on the API key dropdown and select “Generate new key”



5. When prompted, click on Generate then copy the new API key. Paste this into a text document, this will be needed later.

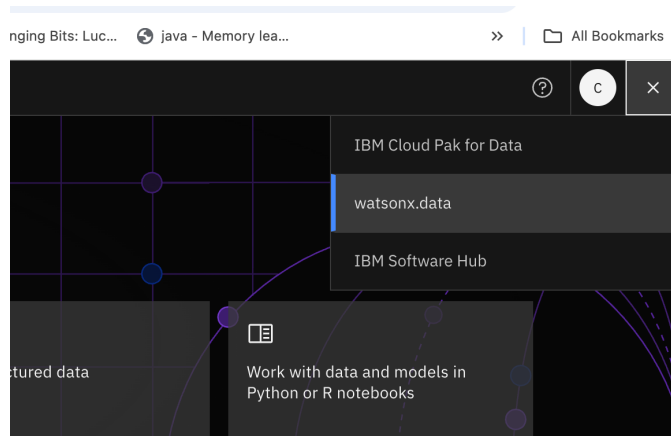
3.2 Set up the project in Cloud Pak for Data

1. Click on the switch locations icon in the top right corner

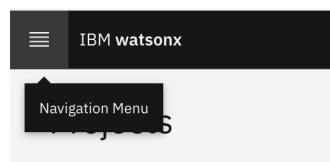


IBM TechXchange

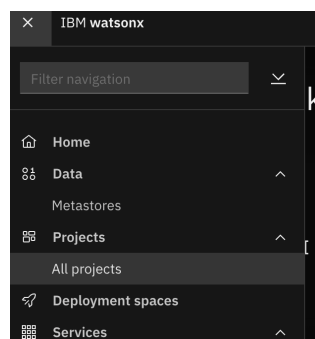
2. Select the watsonx.data context



3. Click on the Navigation Menu in the top left corner



4. Select "All projects" from the menu



5. Click on New and select “Local file”

Create a project

Start with a new, blank project or select from where to import an existing project.



6. Browse and select the file TechXChangeLab2658.zip file (located in Downloads directory), enter a name for the project , example “StudentID_Lab”, then click Create

Create a project

Start with a new, blank project or select from where to import an existing project.

A screenshot of the 'Define details' form in the IBM TechXchange application. The form is titled 'Define details' and has a sidebar on the left with the same menu as the previous screenshot. The main form area contains the following fields: 'Local asset' (with a file icon and the text 'TechXChangeLab2658.zip application/zip'), 'Name' (with a text input field containing 'Student1_lab'), 'Description (optional)' (with a text area containing 'What's the purpose of this project?'), 'Tags (optional)' (with a text input field containing 'Add tags'), and 'Advanced settings' (with a dropdown arrow). At the bottom right, there are two buttons: 'Cancel' (gray) and 'Create' (blue). A small red circular icon with a white 'P' is visible in the bottom right corner of the form area.

7. Verify the project import the project successfully.

[Projects](#) / [Student1_lab](#)

Overview

Assets

Job runs

Manage

Find assets

4 assets

All assets

Asset types

>

Data

4

Data

☐	Name	Last modified
☐	1 config.py text/x-script.python	Now Modified by System
☐	1 initial_loading_all_steps.py text/x-script.python	Now Modified by System
☐	1 medallion.py text/x-script.python	Now Modified by System
☐	1 validate_config.py text/x-script.python	Now Modified by System

IBM TechXchange

3.3 Create environment variables for the spark job

1. Edit the env.sh file in the home directory. VSCode, vi, etc

```
BASE_URL=cpd-cpd.apps.itz-ry66c1.hub01-1b.techzone.ibm.com
ENGINE_ID=spark814
INSTANCE_ID=1788636256617876
VOLUME_NAME="cpd::spark01-volume1"
APP_ENV=onprem
WXD_USERNAME=
WXD_API_KEY=
AE_SPARK_EXECUTOR_COUNT=1
SPARK_DRIVER_CORES=1
SPARK_DRIVER_MEMORY=4G
SPARK_EXECUTOR_CORES=1
SPARK_EXECUTOR_MEMORY=4G
STORAGE_TYPE=s3
SOURCE_CSV=cfpb/complaints.csv
S3_BUCKET_NAME=techxchange-data
S3_SOURCE_BUCKET=techxchange-src
S3_ACCESS_KEY=
S3_SECRET_KEY=
S3_ENDPOINT=https://s3.us-south.cloud-object-storage.appdomain.cloud
MEDALLION_TABLE_FORMAT=iceberg
MEDALLION_DEFAULT_CATALOG=wx_medallion
MEDALLION_BRONZE_SCHEMA=cfpb_bronze
MEDALLION_SILVER_SCHEMA=cfpb_silver
MEDALLION_GOLD_SCHEMA=cfpb_gold
CATALOG_NAME="Medallion Catalog"
CONNECTION_NAME=PrestoConnection
```

2. Change the WXD_USERNAME, WXD_API_KEY, S3_ACCESS_KEY and S3_SECRET_KEY to the values provided.
3. Change the “cfpb_” in the three schema names to match your student number:

```
MEDALLION_BRONZE_SCHEMA=student1_bronze
MEDALLION_SILVER_SCHEMA=student1_silver
MEDALLION_GOLD_SCHEMA=student1_gold
```

3.4 Create the spark jobs

1. Click on the “New Asset” button

[Projects](#) / [Student1_lab](#)

Overview

Assets

Job runs

Manage

▽

Find assets

7 assets

All assets

Asset types

▼ Data 7

All assets

☐	Name	↕	Last modified	
☐	config.py text/x-script.python		52 minutes ago Modified by you	⋮
☐	medallion.py text/x-script.python		52 minutes ago Modified by you	⋮
☐	validate config.py		3 hours ago	⋮

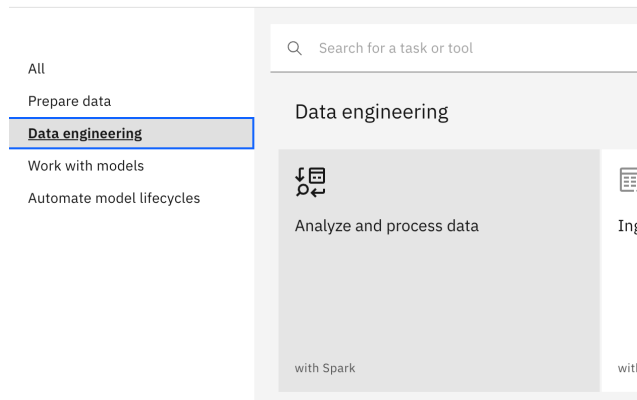
Import assets

New asset +

2. Select Data engineering and Analyze and process data

What do you want to do?

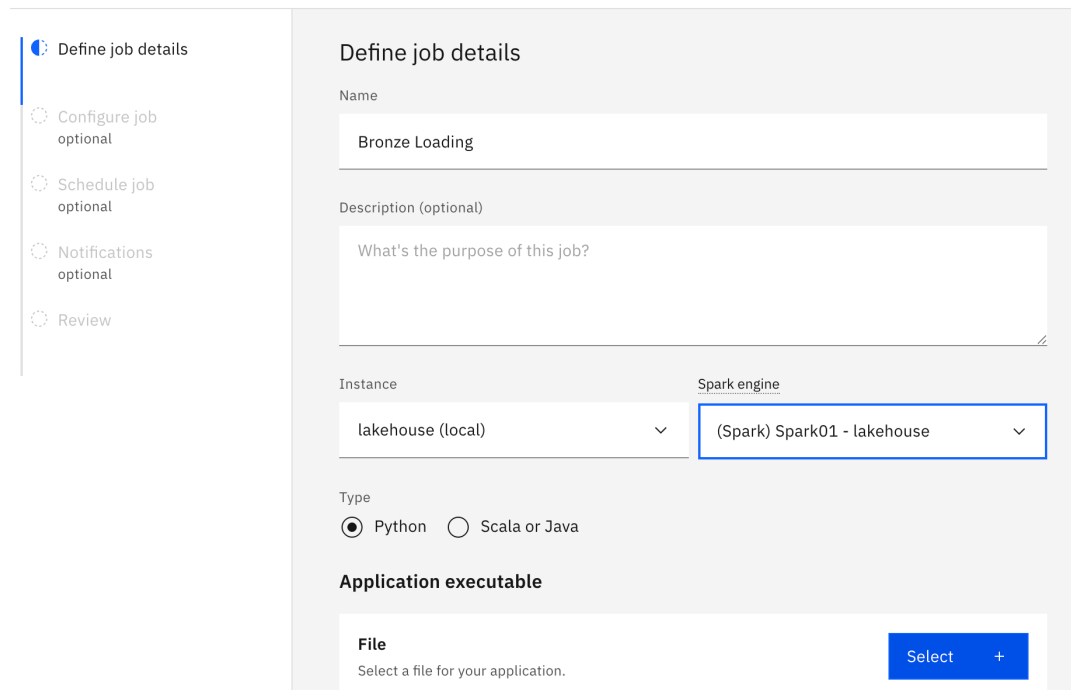
Select a task based on your goal. You'll use a tool to create an asset for that goal.



3. Enter a name for the Bronze loading spark job, ensure a spark engine is selected, then click on Select to choose a spark job file.

Analyze and process data

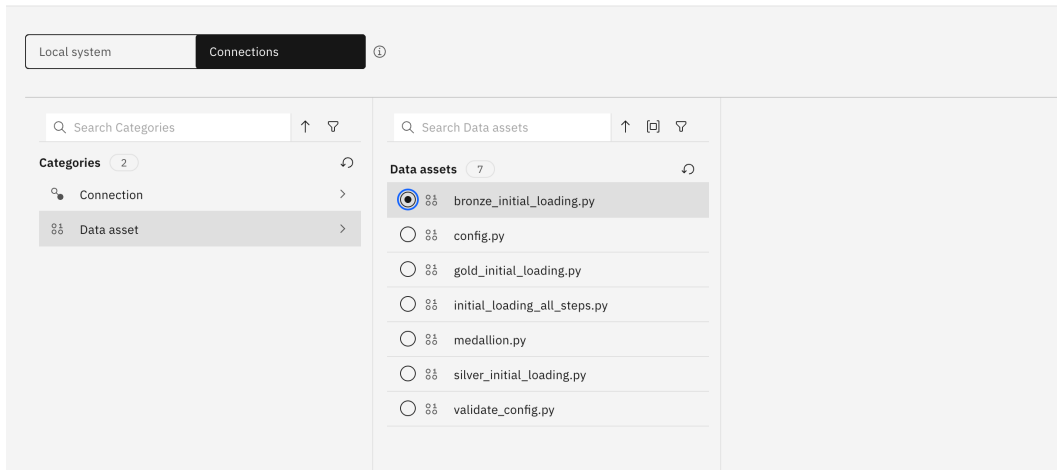
Define the details to create a Spark job.



4. Select Data asset in the category, then click on `bronze_initial_loading.py` from the Data assets then click on the Save button.

Select file

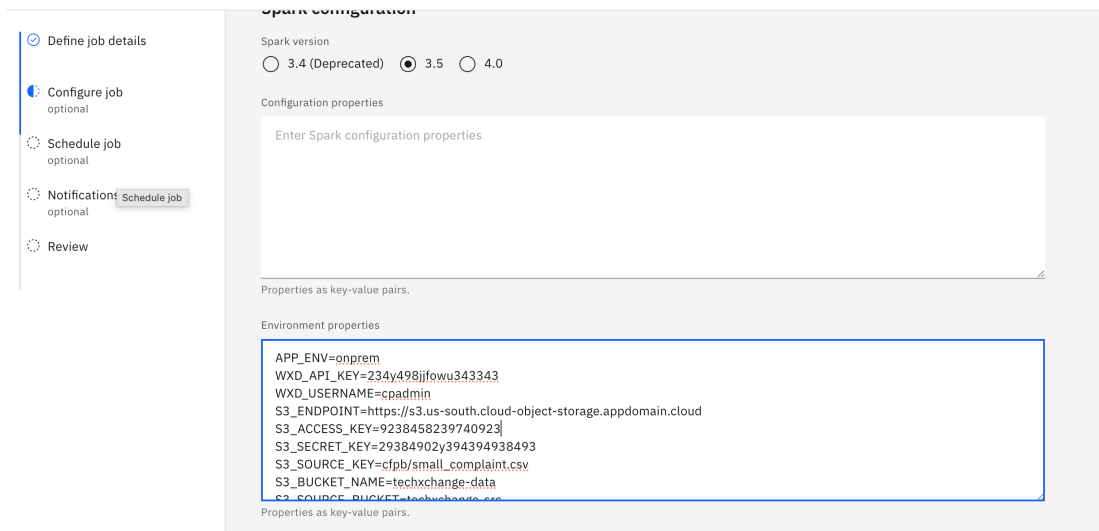
Upload a local file or select a file from your project's connections or data assets.



5. Click on the Next button to get to the job configuration screen and paste in the environment variables.

Analyze and process data

Define the details to create a Spark job.



6. Scroll down and configure the hardware for the spark job (please use the default options)

Hardware configuration

Driver virtual CPU cores

1

-

|

+

Driver memory (GB)

4

-

|

+

Executor virtual CPU cores

1

-

|

+

Executor memory (GB)

4

-

|

+

Number of executors

1

-

|

+

7. Scroll down to Dependencies, open the options and click the Select next to Py-files

Dependencies

Py-files

.zip, .py, or .egg files that need to be placed in the PYTHONPATH for the python application.

Select +

Jars

Jar files that need to be placed in the classpath of the executor and the driver.

Select +

Packages

8. Select the config.py and medallion.py files from the Data asset, then click Save

Select file

Select a file from your project's connections or data assets.

Local system

Connections

①

i For dependencies, you cannot upload files from local system.

Search Categories

↑

▽

Categories 2

↻

Connection

>

Data asset

>

Search Data assets

↑

[🔍]

▽

Data assets 7

↻

☐ bronze_initial_loading.py

☒ config.py

☐ gold_initial_loading.py

☐ initial_loading_all_steps.py

☒ medallion.py

☐ silver_initial_loading.py

☐ validate_config.py

9. The py-files are saved as part of the definition. Click on Next.

^ Dependencies

Py-files

.zip, .py, or .egg files that need to be placed in the PYTHONPATH for the python application.

Add +

medallion.py

Data asset Id: a057daab-ef9c-404d-af6e-472e3e2c4381

×

config.py

Data asset Id: f9270fa1-d86d-4bfb-8c8e-625e86068f3f

×

Jars

Jar files that need to be placed in the classpath of the executor and the driver.

Select +

Packages

Enter packages

10. Unselect the Run after job creation, then click on Next

Analyze and process data

Define the details to create a Spark job.

Define job details

Configure job
optional

Schedule job
optional

Notifications
optional

Review

Schedule job

Run all jobs on demand or set a schedule, or do both.

☒

Run after job creation

☐ Run on a schedule

IBM TechXchange

11. Click on Next 2 more times to finish the wizard. Return to the Assets view to see the result

Projects / Student1_lab

Overview **Assets** Job runs Manage

Find assets Import assets New asset +

8 assets

All assets

Asset types

> Data 7

Jobs 1

Jobs

Name	Job type	Created by	Last modified
Bronze Loading Job	Spark application	Created by you	Now Modified by you

12. Repeat steps 1-11 for silver_initial_loading.py and gold_initial_loading.py named “Silver Loading” and “Gold Loading” respectively.

13. Return to the Assets view to see the result

Projects / Student1_lab

Overview **Assets** Job runs Manage

Find assets Import assets New asset +

10 assets

All assets

Asset types

> Data 7

Jobs 3

Jobs

Name	Job type	Created by	Last modified
Gold Loading Job	Spark application	Created by you	3 hours ago Modified by you
Silver Loading Job	Spark application	Created by you	3 hours ago Modified by you
Bronze Loading Job	Spark application	Created by you	14 hours ago Modified by you

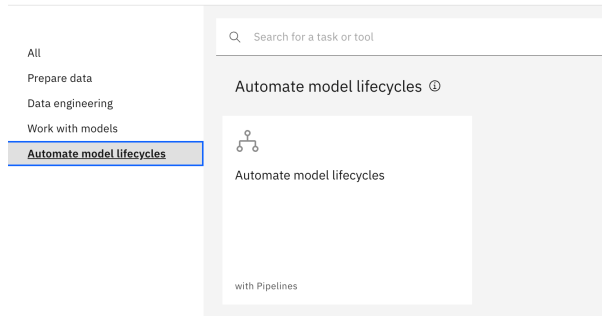
3.5 Create an Orchestration Pipeline for the Spark Jobs

1. From the Assets view in the project, click on the “New Asset” button

2. Select Automate model lifecycles from categories and click on Automate model lifecycles tile

What do you want to do?

Select a task based on your goal. You'll use a tool to create an asset for that goal.



3. Enter a name for the pipeline then click the Create button

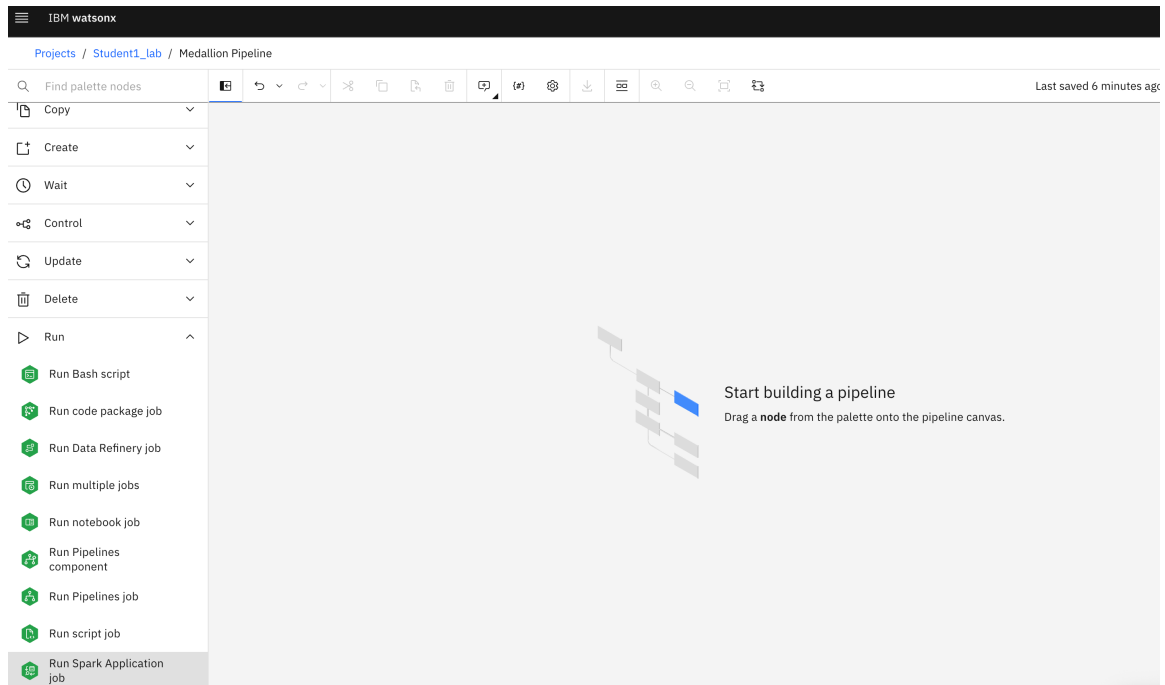
Automate model lifecycles

Define the details to create a Pipeline asset and open it in the Pipelines tool.

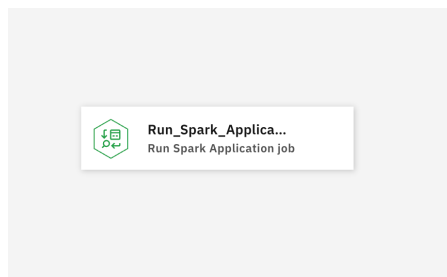
The screenshot shows a form for creating a pipeline. On the left, there is a sidebar with a '+ New' button and a 'Sample' link. The main area is titled 'Define details'. It contains a 'Name' field with the value 'Medallion Pipeline', a 'Description (optional)' text area with the placeholder 'What's the purpose of this Pipeline?', and a 'DataStage settings' toggle switch set to 'Off'. At the bottom right, there are 'Cancel' and 'Create' buttons.

IBM TechXchange

4. Open the Run dropdown and hover over the Run Spark Application job entry



5. Drag the Run Spark Application to the canvas



IBM TechXchange

- Double click on the job on the canvas. Change the Name at the top to BronzeLoading, then click on the Select Job button.

Run Spark Application job

BronzeLoading 

Input

Output

Spark Application Job 

[Search by attributes](#)

Select Job 

Error Policy (optional)

☐ Override [default error policy](#) with:

☒ Fail pipeline on error

☐ Continue pipeline on error

Node ID

run_spark_application_job_3 

- Select the Bronze Loading Job, then click on the Choose button, then click on Save to save the changes.

Select Job

Search Scopes

↑ ▾

Scopes 1

Student1_lab

Search Jobs

↑ ▾

Jobs 3

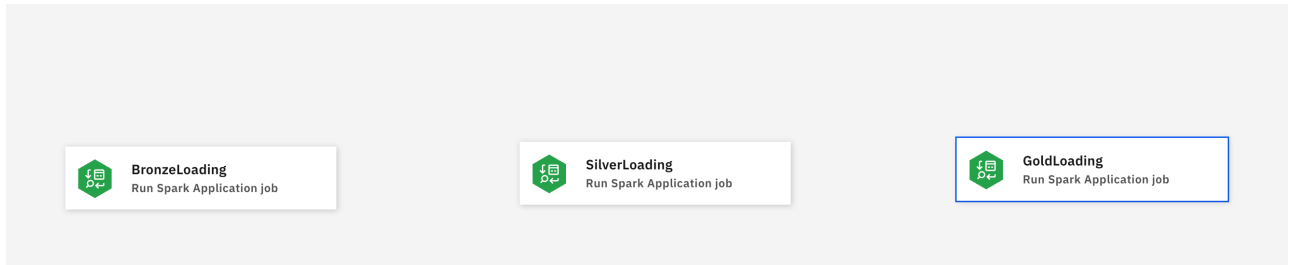
Bronze Loading

Gold Loading

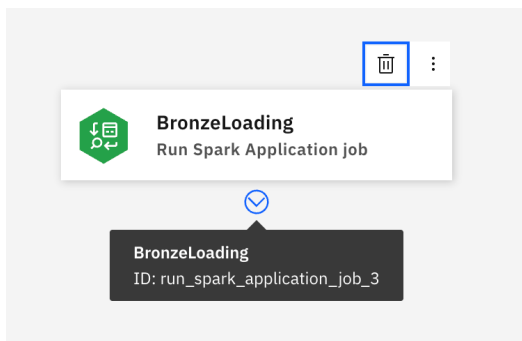
Silver Loading

IBM TechXchange

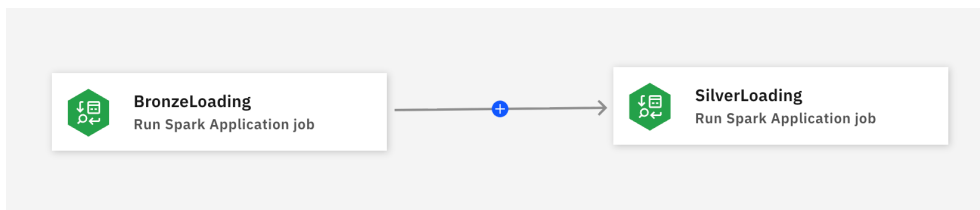
8. Repeat steps 4 through 7 to add jobs for the silver and gold jobs. The canvas should look like the following:



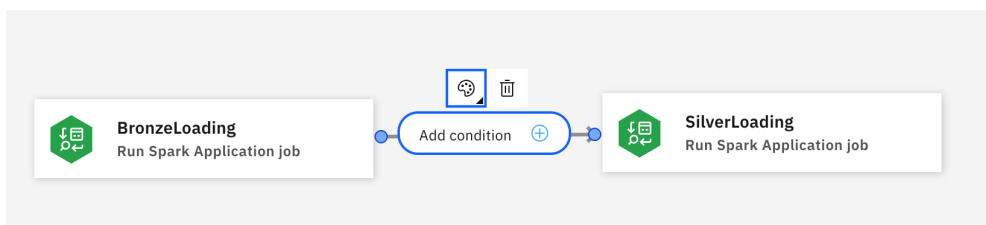
9. Hover over the BronzeLoading until the blue caret is displayed.



10. Drag the caret to connect BronzeLoading to SilverLoading



11. Hover over the blue “+” to add a condition. Click on the Add Condition button.



12. Select “Condition response”, change the Name to completed and select the “Completed” condition for moving to the next step. Click on the Save button.

Conditions

Configure the condition using the options below

Condition type

Condition response ✓
Select status to meet for advancing flow or Otherwise to advance when other conditions are not met

Simple condition
Build a condition using node variables and pipeline parameters

Advanced condition
Build a complex condition using the expression builder

Name

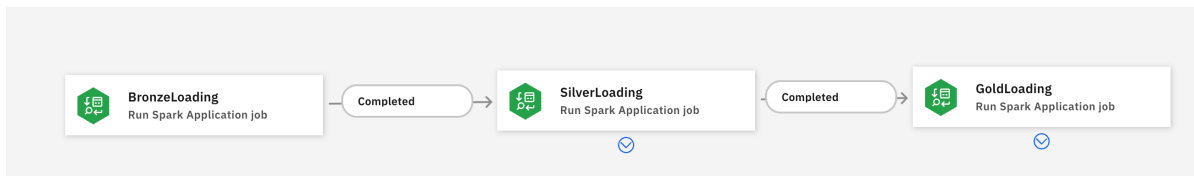
Completed

Condition response

☒ Execute next action when node status is one of these:
☒ Completed
☐ Completed with warnings
☐ Completed with errors
☐ Failed
☐ Cancelled

☐ Otherwise, execute next action when specified conditions for other nodes connected from the same parent are not met

13. Repeat steps 9-12 to connect SilverLoading to GoldLoading.



14. To run the pipeline, click on the Run pipeline button at the top of the canvas.

15. Provide a name for the pipeline run then click on the Run button.

Projects / Student1_lab / Medallion Pipeline / Run tracker

Stop @ Save as job

Running

BronzeLoading Run Spark Applicat... → Completed → SilverLoading Run Spark Applicat... → Completed → GoldLoading Run Spark Applicat...

Consolidated logs

Timestamp	Name	Status	Event description
Aug 20, 2026, 12:25 PM	Medallion Pipeline		Pipeline started (name: Medallion Pipeline, PipelineRun: 3086ca22-25ab-43de-bc2a-5d6bc2a83f8e-9d9ffaa5, Job name: Trial job - Medallion Pipeline, Job ID: c8fa8e53-9d7b-47f1-93d9-d965b282a2a4, Job run ID: 3086ca22-25ab-43de-bc2a-5d6bc2a83f8e)

16. Final State after a successful run

Projects / Student1_lab / Medallion Pipeline / Run tracker

Run again @ Save as job

Completed

- 3 Completed
- 0 Completed with warnings
- 0 Completed with errors
- 0 Skipped
- 0 Failed

BronzeLoading Run Spark Applicat... → Completed → SilverLoading Run Spark Applicat... → Completed → GoldLoading Run Spark Applicat...

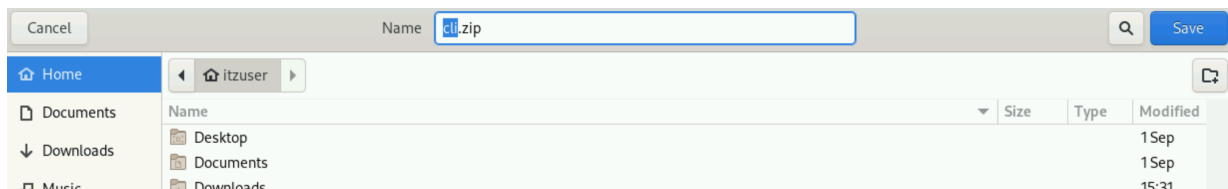
Consolidated logs

Timestamp	Name	Status	Event description
Aug 20, 2026, 2:23 PM	Medallion Pipeline	✓	Pipeline completed (name: Medallion Pipeline, PipelineRun: 6ff9fdae-5bda-4ca5-8bc8-93129f4c36e6-15348bbb, Job name: Trial job - Medallion Pipeline, Job ID: c8fa8e53-9d7b-47f1-93d9-d965b282a2a4, Job run ID: 6ff9fdae-5bda-4ca5-8bc8-93129f4c36e6)
Aug 20, 2026, 2:23 PM	GoldLoading	✓	Job run 602e83f5-362a-44a8-84ea-35af1886a47a completed with status: Completed
Aug 20, 2026, 2:21 PM	Medallion Pipeline	✓	Node completed (Node ID: cel-link-run-spark-application-job-5-0)
Aug 20, 2026, 2:21 PM	SilverLoading	✓	Job run 31739f07-aefd-466c-b0c1-5a3a0e7dbaae completed with status: Completed

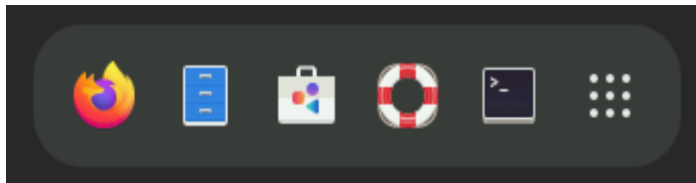
4 Load Medallion Data using CLI tools (Optional)

4.1 Configure the cli directory

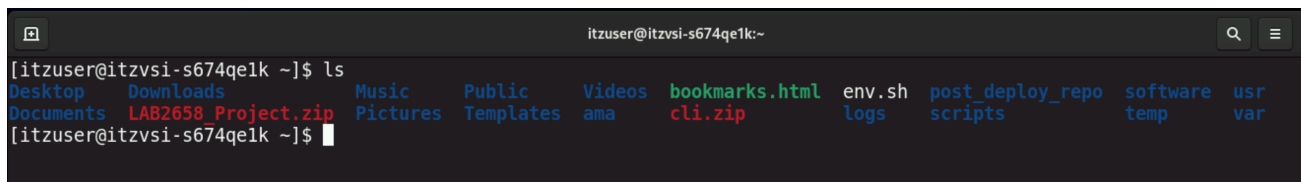
1. Complete the env.sh setup from section 3.3 above.
2. Open the github link and save the cli.zip file to the Home directory.



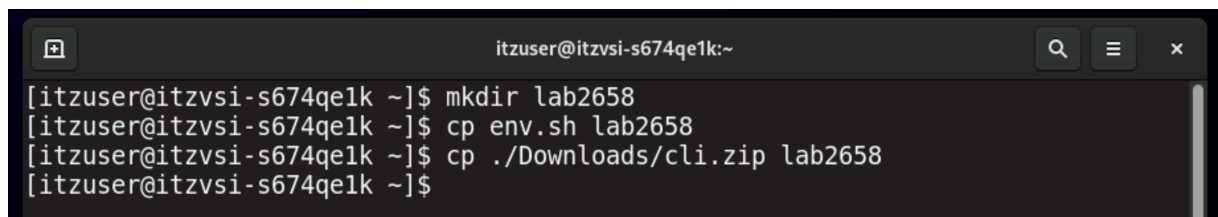
3. From the activities icons, open a new terminal



4. Execute an “ls” to ensure the downloaded files are present



5. Create a directory named lab2658 and copy the env.sh and cli.zip. Note: cli.zip is located in the Downloads directory. Note: cli.zip is in the Downloads directory



IBM TechXchange

6. Change directory to lab 2658 and unzip the cli.zip

```
itzuser@itzvsi-s674qe1k:~/lab2658
[itzuser@itzvsi-s674qe1k ~]$ cd lab2658/
[itzuser@itzvsi-s674qe1k lab2658]$ ls
cli.zip  env.sh
[itzuser@itzvsi-s674qe1k lab2658]$ unzip cli.zip
Archive:  cli.zip
  creating: cli/
  inflating: cli/getApplication.sh
  inflating: cli/bronze_initial_loading.sh
  inflating: cli/all_steps_initial_loading.sh
  inflating: cli/gold_initial_loading.sh
  inflating: cli/submitJob.sh
  inflating: cli/getLogs.sh
  inflating: cli/validateConfiguration.sh
  creating: cli/jobs/
  inflating: cli/jobs/config.py
  inflating: cli/jobs/medallion.py
  inflating: cli/jobs/gold_initial_loading.py
  inflating: cli/jobs/silver_initial_loading.py
  inflating: cli/jobs/all_steps_initial_loading.py
  inflating: cli/jobs/bronze_initial_loading.py
  inflating: cli/jobs/validate_config.py
  inflating: cli/getToken.sh
  inflating: cli/silver_initial_loading.sh
[itzuser@itzvsi-s674qe1k lab2658]$
```

7. Copy env.sh into the cli directory

```
itzuser@itzvsi-s674qe1k:~/lab2658
[itzuser@itzvsi-s674qe1k lab2658]$ ls
cli  cli.zip  env.sh
[itzuser@itzvsi-s674qe1k lab2658]$ cp env.sh ./cli
[itzuser@itzvsi-s674qe1k lab2658]$
```

8. Change directory to the cli directory

```
itzuser@itzvsi-s674qe1k:~/lab2658/cli
[itzuser@itzvsi-s674qe1k lab2658]$ cd cli
[itzuser@itzvsi-s674qe1k cli]$ ls
all_steps_initial_loading.sh  gold_initial_loading.sh
bronze_initial_loading.sh    jobs
env.sh                        silver_initial_loading.sh
getApplication.sh             submitJob.sh
getLogs.sh                    validateConfiguration.sh
getToken.sh
[itzuser@itzvsi-s674qe1k cli]$
```

IBM TechXchange

9. Execute the `getToken.sh` command. Note: This must be executed as “`./getToken.sh`”

```
$ ./getToken.sh
Bearer Token created successfully
```

10. The bearer token is stored as an environment variable. Execute the “`./bronze_initial_loading.sh`” command

```
$ ./bronze_initial_loading.sh
Bronze Loading Job created successfully
```

11. Execute the “`./getApplication.sh`” script

```
$ ./getApplication.sh

{"spark_application_name": "Medallion Initial Loading",
 "spark_version": "3.5",
 "start_time": "Sunday 13 September 2026 21:33:20.331+0000",
 "state": "RUNNING",
 "submission_time": "Sunday 13 September 2026 21:33:00.217+0000",
 "wxd_application_ui_endpoint": "https://cpd-cpd.apps.wx-data-dev.ibm-cpd-
partnerships.com/lakehouse/api/v3/1788371601764706/spark_engines/spark619/applications/c5071b8e-9ee8-
4154-b89b-5ffd21db32a2/ui"
}
```

12. Once the state is in `RUNNING`, the “`./getLogs.sh`” script can be run to see logs at any time. This script uses the current running job ID to show the logs reflecting the currently running job.

```
$ ./getLogs.sh

Count of records in the wx_medallion.cfpb_bronze.summary_by_year table
+-----+
|count(1)|
+-----+
|      16|
+-----+

Bronze Loading Completed
```

13. Run the “./getApplication.sh” script until the state is “FINISHED”

```
$ ./getApplication.sh

  "id": "c5071b8e-9ee8-4154-b89b-5ffd21db32a2",
  "return_code": "0",
  "spark_application_id": "app-c5071b8e-9ee8-4154-b89b-5ffd21db32a2-1789335199603",
  "spark_application_name": "Medallion Initial Loading",
  "spark_version": "3.5",
  "start_time": "Sunday 13 September 2026 21:33:20.331+0000",
  "state": "FINISHED",
  "submission_time": "Sunday 13 September 2026 21:33:00.217+0000"
}
```

14. Execute the “./silver_initial_loading.sh” command

```
% ./silver_initial_loading.sh
Silver Loading Job created successfully
```

15. Run the “./getApplication.sh” script until the state is “FINISHED”. The “./getLogs.sh” can be run when state is in “RUNNING”

```
$ ./getApplication.sh

  "return_code": "0",
  "spark_application_id": "app-fb4e9cdb-be98-4972-a73e-dba82cca9c59-1789336145320",
  "spark_application_name": "Medallion Initial Loading",
  "spark_version": "3.5",
  "start_time": "Sunday 13 September 2026 21:49:05.988+0000",
  "state": "FINISHED",
  "submission_time": "Sunday 13 September 2026 21:48:45.999+0000"
}
```

16. Execute the “./gold_initial_loading.sh” command

```
$ ./gold_initial_loading.sh
Gold Loading Job created successfully
```

17. Run the “./getApplication.sh” script until the state is “FINISHED”. The “./getLogs.sh” can be run when state is in “RUNNING”

```
$ ./getApplication.sh

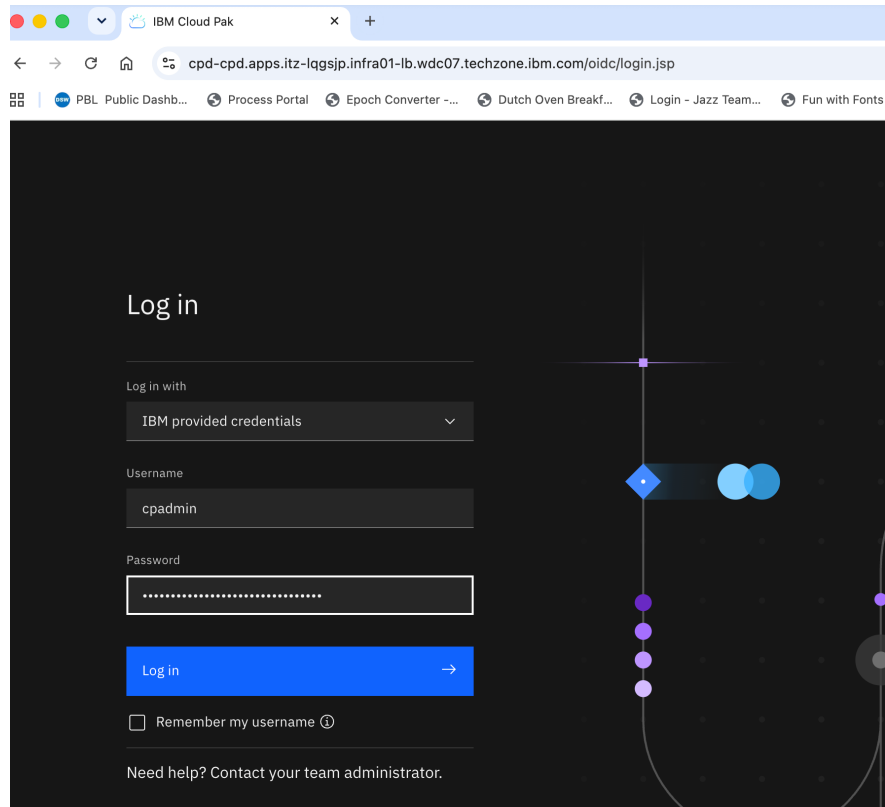
  "return_code": "0",
  "spark_application_id": "app-fb4e9cdb-be98-4972-a73e-dba82cca9c59-1789336145320",
  "spark_application_name": "Medallion Initial Loading",
  "spark_version": "3.5",
  "start_time": "Sunday 13 September 2026 21:49:05.988+0000",
  "state": "FINISHED",
  "submission_time": "Sunday 13 September 2026 21:48:45.999+0000"
}
```

5 Verify the Medallion Tables

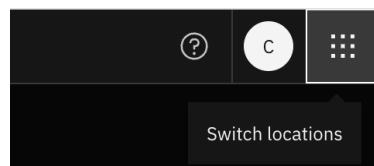
5.1 Verify one of the medallion tables

1. Log in to cloud pak for data

<<Place link here>>, use creds from cheat sheet

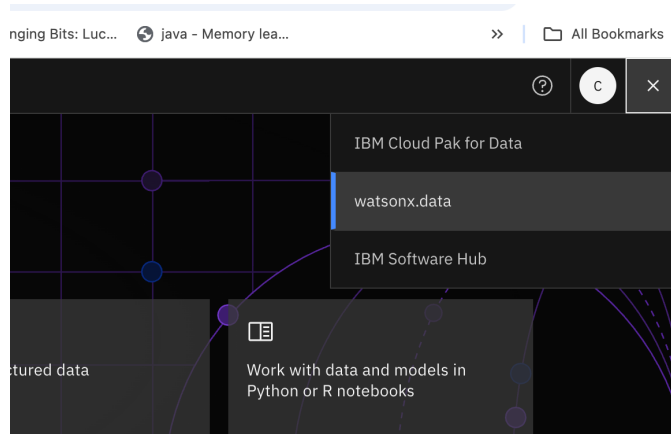


2. Click on the switch locations icon in the top right corner

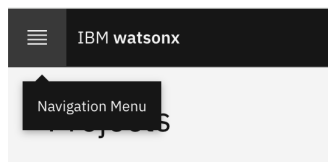


IBM TechXchange

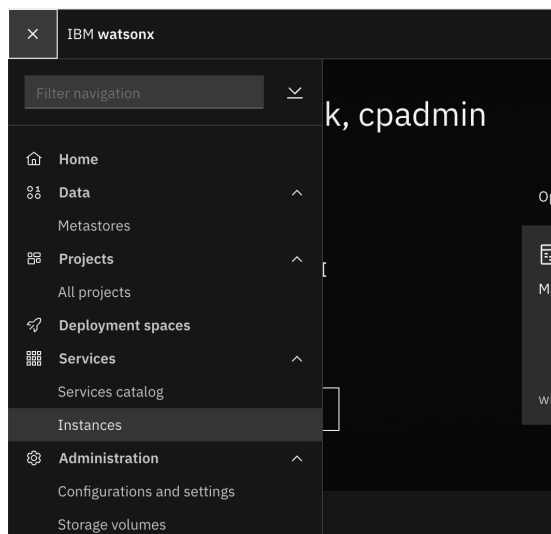
3. Select the watsonx.data context



4. Click on the Navigation Menu in the top left corner



5. Click on the menu dropdown in the top left corner of the page. Select Instances from the menu.



6. Click on the open icon on the right side of the screen for the lakehouse entry.

Service instances

Work with the service instance that you have access to or create new service instances.

Last updated: 8/26/2026 5:31 PM ⓘ

Find Instances

⌵

⌵

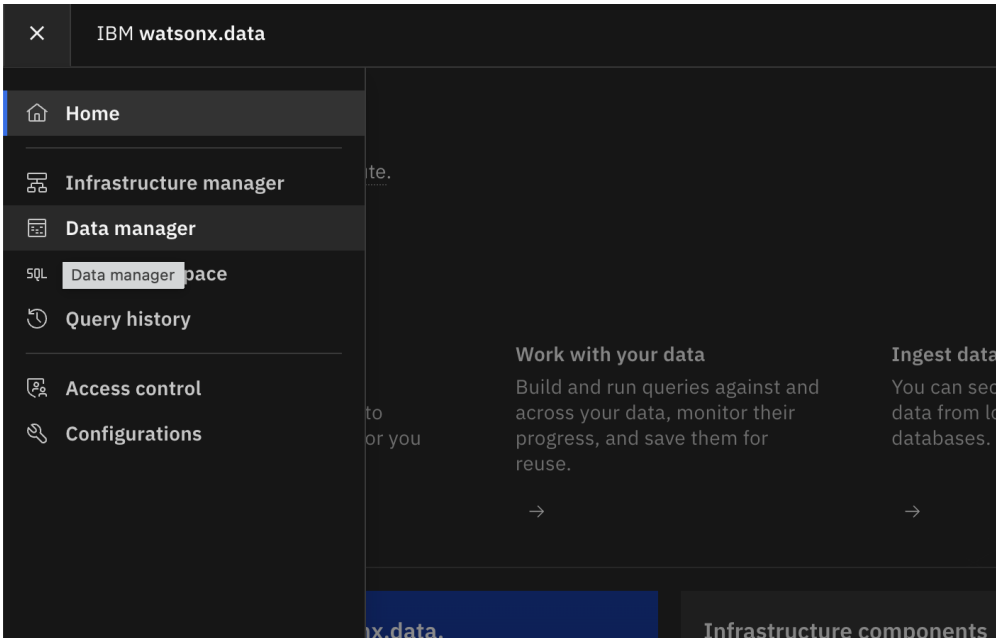
⌵

New instance +

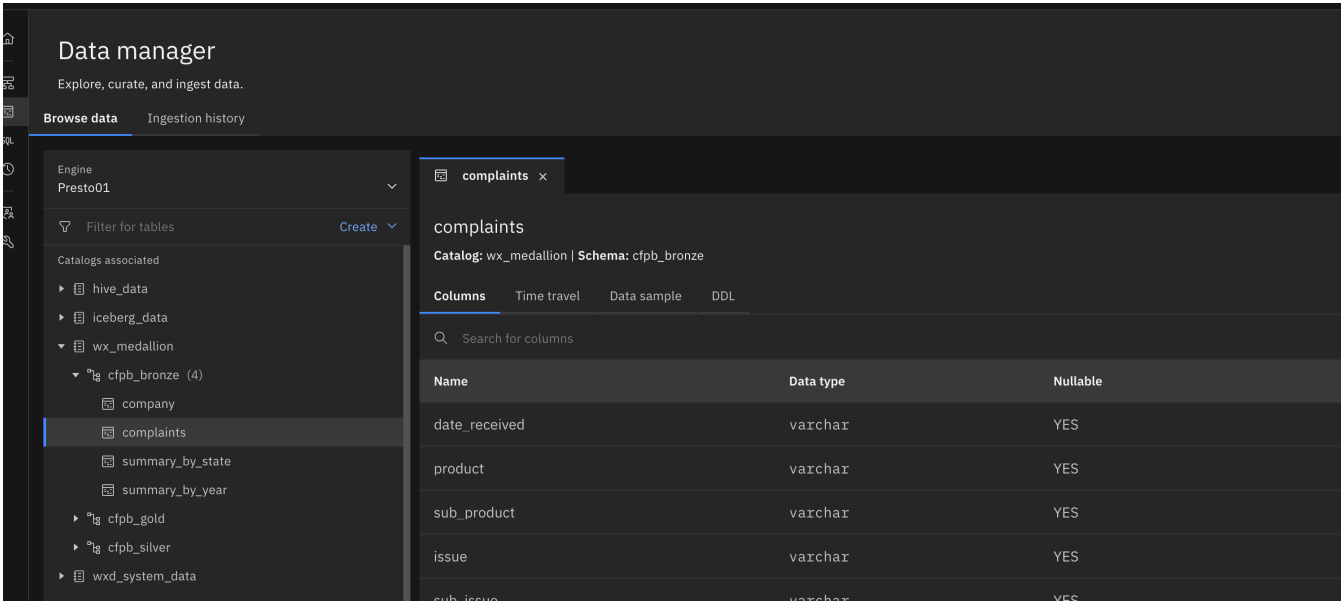
Name	Type	Data plane	Physical location	Created by	Created on	
ProfHbIntrnl	spark	—	—	__internal_profiler__	Aug 14, 2026	⋮
lakehouse IBM watsonx.data	watsonx-data	—	—	cpadmin	Aug 14, 2026	⋮

Open

7. The lakehouse screen will open in a new window. Click on the Data Manager icon on the left side of the screen.



8. When the data manager opens, click on the wx_medallion twist, the click on the bronze schema, then the complaints table.



9. Clicking on the Data sample tab will show a small subset of records

The screenshot shows the 'complaints' table with the 'Data sample' tab selected. The table displays a subset of records with columns: 'date_received', 'product', 'sub_product', 'issue', 'sub_issue', 'complaint_what_happened', 'company_public_response', and 'company'. The 'sub_product' column is highlighted with a dropdown arrow. The records show various complaint types such as 'Vehicle loan', 'Mortgage', 'Credit card', and 'Mortgage'.

date_received	product	sub_product	issue	sub_issue	complaint_what_happened	company_public_response	company
2018-02-27	Vehicle loa...	Loan	Managing t...	Problem wi...	null	null	CAPITAL O...
2015-05-04	Mortgage	Other mort...	Loan servic...	null	null	null	Ocwen Fin...
2014-04-21	Credit card	null	Other	null	null	null	AMERICAN...
2019-07-26	Mortgage	Convention...	Struggling t...	null	null	null	JPMORGA...
2019-05-22	Credit repo...	Credit repo...	Incorrect i...	Informatio...	null	Company has responde...	TRANSUNI...
2014-10-30	Mortgage	Convention...	Loan servic...	null	null	null	Ocwen Fin...
2015-02-21	Mortgage	Convention...	Trouble dur...	Fees charged	null	null	Mr. Casper

6 Automated Registration of Medallion Tables with IBM Knowledge Catalog (Demo)

7 Confluent Integration with Medallion and OpenSearch (Demo)

8 Feedback section

Use this section to provide feedback on the lab.

9 Getting help and troubleshooting

This section provides information about getting help with your lab and some common troubleshooting topics.

9.1 Reach out to the Lab instructor.

9.2 Common troubleshooting tips

Use this section to document common troubleshooting issues that users may experience.

9.3 Getting help

Customize this section as needed, keeping in mind that some individuals will be doing this lab in the self-paced room without an instructor and would need guidance on where to go for assistance.

10 Lab Reference Materials (for instructor)

Provide links to products, documentation on other references for students to continue their education.

Provide links to TechXchange 2026 documentation / Lab download site.